Projet CellStriker

Reproduction du jeu cell Machine

Rapport de projet - Programmation en Java

Groupe :

- Hugo CALDIER
- Elias DAI
- Daniel DULNIK
- Ethan DUQUENNOY
- Baptiste LAMOUROUX

Encadrant : Vincent PADOVANI

Année Universitaire

2024 - 2025
Licence informatique fondamentale
UE : Projet de programmation

Sommaire

Introduction 3
Sujet du projet 3
Liste des cellules 3
Fonctionnalité de CellStriker 4
Nouvelles cellules 4
Conception du projet 5
Arborescence des fichiers 5
Diagramme des classes 6
Choix d'implémentation et fonctionnement 7
Modèle 7
CellType 7
Direction et Position 7
Cell et Grid 8
Fonctionnement détaillé des cellules 8
Vue 10
Menus 10
GridView, Editor et Sandbox 11
ImageLoader 11
Texture, Render 11
RetroGlassPane 12
AnimaCell 12
Contrôleur 12
GridController, LevelEditorController et SandboxController 12
Interaction et InteractionSandbox 13
Gestion des niveaux 13
LevelLoader 13
LoaderJson 13
Gestion de l'audio 14
Audio et playlist 14
Les modes de jeux 15
Mode spécial 15
Editeur de niveau 15
Sandbox 15
Hitman 16
Classe principale 16
Game 16
Screen 16
Conclusion 17
Répartition du groupe 17
Problème et solution 19
Conclusion 20

Introduction

Sujet du projet

Le projet est une reproduction du jeu *Cell machine,* un jeu qui permet de jouer à différents casse-tête sur des grilles de cellules rectangulaires. La grille possède des zones de construction et des cellules qui doivent être déplacées pour détruire les cellules ennemies et finir le niveau.

Le jeu se déroule en 2 phases, une phase de préparation ou le joueur va placer les cellules afin de résoudre le niveau et une phase d'exécution ou les comportements des cellules vont être activé et le jeu va jouer automatiquement.

Liste des cellules

- Move : se déplace dans la direction pointée, peut pousser des cellules autre que le Wall
- Enemy : cellule à détruire en rentrant en collision avec une cellule
- Wall, Directional, Slide, Push : des blocs immobiles, déplaçables respectivement : dans aucune direction, seulement vers une direction, seulement horizontalement / verticalement et dans toutes les directions.
- Spinner : permet de faire tourner dans le sens horaire ou anti-horaire
- Generator : duplique devant elle la cellule placée derrière, elle peut pousser vers l'avant ou vers l'arrière en fonction des situations.
- Trash : supprime la cellule qui rentre en collision avec.

Fonctionnalité de CellStriker

Le jeu va donc implémenter les fonctionnalités provenant du jeu original, on pourra donc retrouver des niveaux jouables avec les 10 cellules de base du jeu et différents menus pour changer des paramètres ou bien choisir un niveau.

Les fonctionnalitées ajoutées pour notre jeu permettent un amélioration de l'expérience utilisateur, comme un ajout d'un mode éditeur afin de permettre au joueur la création de niveau unique, un mode sandbox pour permettre de pratiquer différents tests sur les cellules du jeu, un mode Hitman afin de jouer d'une autre manière au jeu, une plus grande variété de niveau avec des niveaux de trouvant dans le mode spécial où l'on peut changer les règles de base du jeu. De nouvelles cellules ont aussi été rajoutées pour fournir des casse-têtes plus innovants et complexes .

Nouvelles cellules

- Wanderer : une cellule ennemie qui n'est plus immobile mais dynamique selon un algorithme.
- Repulse : repousse les cellules adjacentes à elle.
- Switcher : échange les cellules dans la direction ou elle pointe.
- Nudge : même comportement que le Mover mais ne peut pas détruire un ennemi mais le déplacer, peut aussi déplacer une Trash
- Hitman: La cellule Hitman ne se déplace pas d'elle-même mais elle peut être poussée. C'est la seule Cellule qui peut détruire la Cellule Boss au contact. Elle ne peut être tuée que par une Trash.
- Boss: Cellule ennemie qui ne peut être éliminée qu'au contact d'un Hitman, ou d'une Trash (bien que cela n'est pas réalisable)

Le jeu possède aussi différentes musiques et bruitages en lien avec les mouvements des cellules et des animations qui les accompagnent pour rendre une expérience agréable et plus claire pour le joueur.

Conception du projet

Arborescence des fichiers

```
\---src
    +---libs
    |   \---json
    +---main
    |   \---java
    |       \---com
    |           +---audio
    |           +---controllers
    |           +---jsons
    |           |   +---editor
    |           |   +---levels
    |           |   |   +---hitmanLevels
    |           |   |   +---regularLevels
    |           |   |   \---specialLevels
    |           |   \---sandbox
    |           +---level
    |           +---models
    |           \---views
    |               +---effects
    |               +---menu
    |               |   +---option
    |               |   \---saveLoad
    |               +---sandboxEditor
    |               \---specialMode
    \---resources
        +---background
        +---soundtrack
        \---texture
```

Diagramme des classes

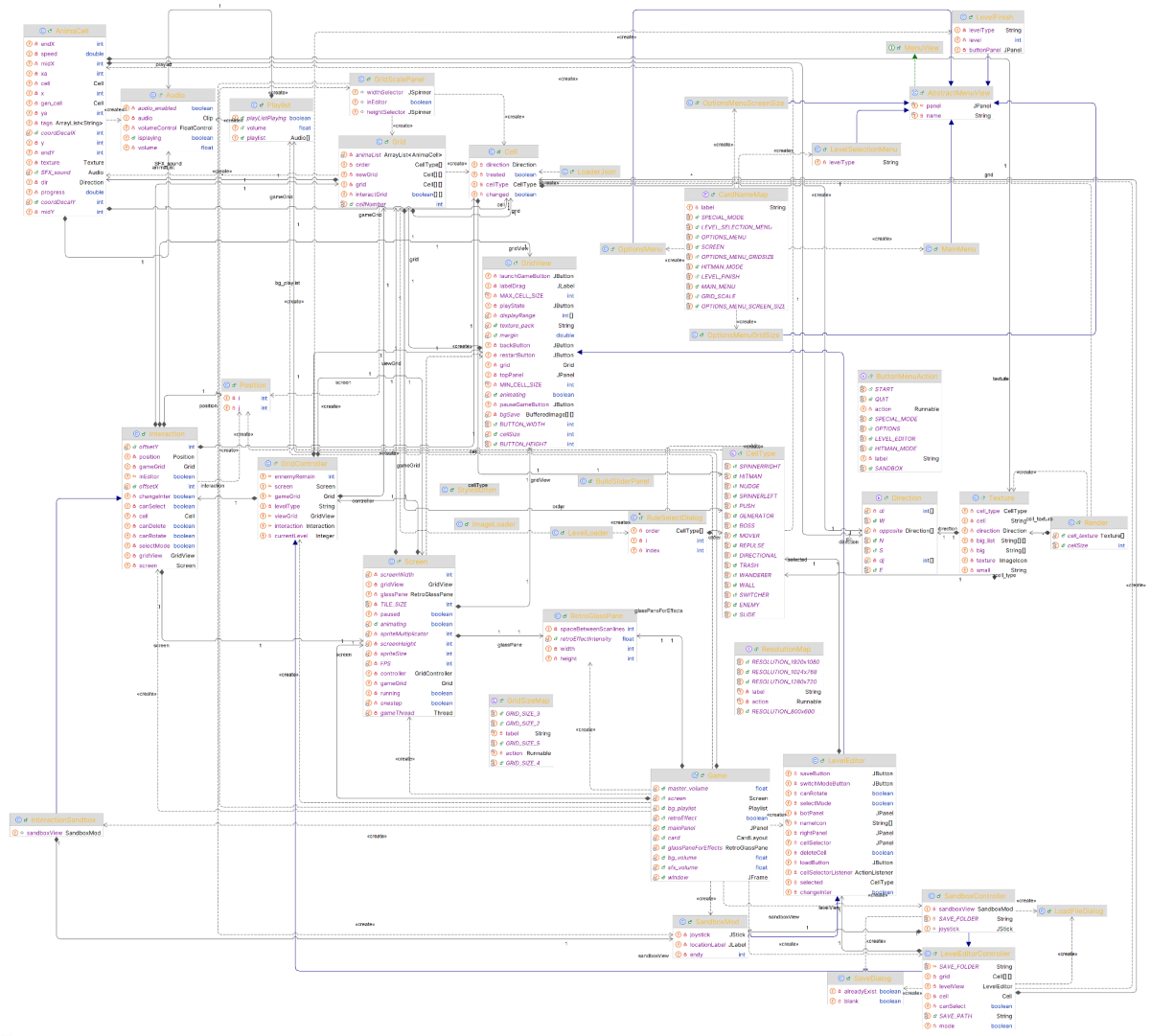

Diagramme fourni dans le dépôt gitlab en meilleur qualité.

Choix d'implémentation et fonctionnement

Le projet a été réalisé en suivant une architecture MVC afin de séparer les différents fonctionnements du jeu. Nous avons donc séparé en différents répertoires comme vu dans l'arborescence.

Quelques design pattern ont aussi été implémentés pour mieux structurer le code : le Game State Pattern et le Factory Pattern.

Modèle

Ce répertoire contient le cœur du jeu avec la définition des comportements des cellules, le système de direction et de position mais aussi le fonctionnement principale de la grille du jeu pour activer la phase d'exécution en jeu.

CellType

Permet l'implémentation des comportements, le choix d'une énumération a été fait car les comportements des cellules sont constants et ne doivent pas varier pendant le jeu. On y retrouve la fonction principale *update* qui permet le calcul du prochain état d'une cellule, chaque cellule va donc redéfinir sa fonction *update* afin de définir son comportement à calculer, de plus une fonction *canMove()* permettant de déterminer si une cellule peut se déplacer vers une direction donnée sans prendre en compte son environnement peut être redéfini pour les cellules dynamiques et il y a aussi une fonction pushCell qui permet de pousser une rangée de cellules vers une direction donnée si et seulement si bien sûr chacune de ces cellules sont en capacité de se déplacer.

Direction et Position

Les directions sont définies par une énumération car elles doivent aussi rester constantes en fonction du temps et permettent différents calculs comme une fonction pour tourner, renvoyer l'opposé ou autres.
Les positions sont elles définies par une classe car elles doivent varier en fonction des différentes instances, elles permettent d'encapsuler des coordonnées i,j pour réaliser différents calculs et changement de position.

Cell et Grid

Cell est donc une classe qui permet l'encapsulation des informations des cellules donc leur type, leur direction et deux booléens pour voir leurs différents changements. Elle possède une fonction principale *update* qui permet l'appel de la fonction *update* de leur type *CellType*.

La classe Grid représente la grille du jeu. Elle contient la grille de cellule de l'état actuel, la grille de cellule du prochain état, un double tableau de booléen informant quelles sont les cases de la grille avec lesquelles l'on peut interagir et un tableau de type de cellule donnant une information sur l'ordre de priorité actuel des cellules, il est modifiable pour changer l'ordre de priorité si nécessaire (ce qui est le cas dans le mode spécial).

Elle contient la méthode updateAll() permettant de lancer un cycle de jeu et de changer l'état de la grille actuelle selon les cellules présentes dans la grille. Son fonctionnement est le suivant:

- On parcours la tableau de l'ordre de priorité des cellules
- On parcourt chacune des cases de la grille
- Si la case où l'on est correspond à une cellule. On vérifie si son type correspond à celui (à compléter). On l'a marque comme étant traité. On appelle sa méthode update et on échange la grille du prochain état avec celle de l'état actuel.
- À la fin on parcourt toute la grille de cellule pour marquer toutes les cellules ayant été traitées comme non traitées.

La classe Grid contient une sous-classe nommée GridCheckers, qui possède des fonctions capables de nous renvoyer différentes parties du voisinage des cellules à partir de leur position.

Fonctionnement détaillé des cellules

Les cellules n'ayant aucun impact direct sur les cellules environnantes ne sont pas considérées comme des cellules "actrices". Elles ne redéfinissent pas la méthode update et ne sont pas appelées dans Grid pour les faire fonctionner.

Les cellules "actrices" utilisent les "checker" GridChecker pour analyser leur environnement et l'impacter.

- Un spinner fait changer la direction des cellules autour de lui, il utilise la méthode checkNeighbor pour les récupérer.
- Le générateur regarde si dans sa direction opposée il y a une cellule à générer. Si c'est le cas il utilise pushCell pour pousser les cellules se trouvant devant lui et placer le clône ou pour déterminer s' il peut en pousser. S' il n'a pas pu placer sa copie, il pousse cette fois-ci les cellules se trouvant dans sa direction opposée.
- Le Mover fait deux vérifications : il vérifie si il a assez de puissance pour pousser les cellules se trouvant devant lui et il vérifie si les cellules se trouvant devant lui peuvent bouger. Pour la première vérification il compte les mover allant dans la même direction que lui qui se trouve dans la chaîne de cellule de sa direction opposée et il compte également ceux se trouvant devant lui. En comptant ceux de trouvant derrière lui il regarde si il a au moins un mover se trouvant dans sa direction opposée qui va dans la même direction que lui et qui peut bouger la chaîne de cellule, si c'est le cas il ne poussera pas la chaîne de cellule car c'est ce mover là qui le fera.
- Il arrête de compter s' il trouve un mover allant dans une direction opposée. Pour la deuxième vérification, il vérifie à l'aide de la méthode canMove de Grid si la chaîne de cellule est en capacité de bouger.
- Le repulse a un fonctionnement très simple. Il appelle pushCell quatre fois pour pousser les quatres rangées de cellules potentielles.
- Le nudge vérifie d'abord si il y a des nudges se trouvant dans la chaîne de cellule dans sa direction opposée qui vont dans la même direction que lui. Si c'est le cas, il ne fait rien sauf si ce nudge là ne peut pas pousser la rangée de cellules. Il vérifie ensuite s' il peut pousser les cellules se trouvant devant lui puis s' il le peut il les pousse.
- Le switcher échange juste les cellules se trouvant sur les cases qui sont dans sa direction et dans sa direction opposée.
- Le wanderer dans un premier temps va voir s' il peut se déplacer vers la case qui se trouve devant lui et si dans cette même direction un mover ne lui arrive pas dessus. Si ce dernier cas survient, le wanderer va chercher à aller dans la case libre la plus proche en commençant à chercher chacune des directions en commençant par le nord dans le sens des aiguilles d'une montre. Dans le

premier cas, il va alors s'orienter vers la direction lui permettant de parcourir la plus longue distance et commencer à s'y déplacer sauf si ce chemin contient un mover ne déplaçant pas de rangé de cellules qui va en direction du wanderer.

- Les cellules ennemies et les cellules pouvant supprimer d'autres cellules n'agissent pas. En fait, dans la méthode move de Grid des conditions permettent que lorsqu'une cellule se déplace sur une de ces cellules elle soit supprimée et que si cette cellule est un ennemi, qu'elle soit aussi éliminer.

Vue

Ce répertoire concerne l'affichage graphique du jeu, elle est actualisé principalement par le changement de contenu dans le modèle donné par le contrôleur. Les différents menus et affichage sont implémentés grâce aux librairies Java Swing / AWT.

Menus

Les différents menus du jeu ont été créés en utilisant un template qui est la classe *AbstractMenuView* qui implémente l'interface *MenuView* qui possède les 2 fonctions primordiales pour les menus. Le template possède donc des fonctions qui définissent un comportement de base pour les menus, on peut aussi redéfinir certaines fonctions pour changer le titre, le layout du panel et le panel principal, il y a aussi une fonction abstraite qui est la fonction principale et qui doit être écrite afin de créer le contenu à mettre dans le menu (bouton, affichage…). Une sous-classe est aussi dans le template, elle permet la création d'un panel possédant un fond spécifique avec une redéfinition de la fonction *paintComponent* pour l'adapter à la taille de l'écran, il est utilisé pour éviter une redéfinition pour chaque menu d'un panel possédant ce fond. Tous les menus principaux redéfinissent donc les fonctions héritées du template, ce qui réduit le code redondant. On peut aussi mentionner certains menus comme *MainMenu, OptionsMenuGridSize et OptionsMenuScreenSize* qui possède des énumérations qui leur sont propre et qui stocke les boutons qu'ils doivent afficher, avec leur label et leur fonction, ces choix d'énumérations ont été fait pour une meilleure

modularité de ces menus (ajout de bouton rapide) et aussi une factorisation de la redondance pour la création de plusieurs boutons.

GridView, Editor et Sandbox

GridView est la classe principale pour afficher la grille du jeu, elle est utilisée lorsque le joueur lance un niveau. La classe hérite de la classe JPanel de Swing, cette classe redéfinie donc la méthode *paintComponent* cette méthode utilise les fonctionnalitées d'un Graphics2D afin d'appliquer des renderingHints pour améliorer la qualité de l'image et afficher les textures des cellules avec l'aide de transformation affine, lorsque l'image correspondante à la cellule est récupérée, la taille de l'image est calculée avec sa position pour les utiliser dans la transformation affine pour ensuite dessiner l'image sur le Graphics2D correspondant au panel de la grille. Lorsque la fonction *paintComponent* il y a aussi un appel au fonction liée à l'animation pour les cellules concernées.

Les classes Editor et Sandbox sont donc des classes héritières de la classe GridView car elles doivent juste changer la manière que le joueur a pour interagir. Sandbox est plus précisément héritière d'Editor.

Concernant l'affichage de l'animation pour dessiner la cellule qu'on attrape pendant la phase de préparation est une animation de *Drag and drop,* les informations pour afficher la cellule sont envoyées par le contrôleur (GridController) et elles sont affichées par les fonctions *startDrag, updateDrag et stopDrag*.

ImageLoader

La classe ImageLoader contient des méthodes permettant de charger des images, de les tourner et de les redimensionner. Elle existe à des fins de factorisation du code.

Texture, Render

Les classes Texture et Render permettent la gestion des textures pour mieux les manipuler. Render permettait la gestion de l'affichage ASCII créé pour le début de la création du jeu.

RetroGlassPane

Cette classe permet de rajouter des effets rétro sur le jeu pour le rendre plus dans le style arcade. Elle possède 2 fonctions principales qui permettent de dessiner ces effets. Ce panel est utilisé ensuite dans la classe Screen pour dessiner par-dessus l'image du jeu les effets désirés.

AnimaCell

La classe AnimaCell est celle s'occupant du traitement des animations des cellules. Chaque instance stocke toutes les informations relatives à l'animation que l'on veut éxecuter comme par exemple les coordonnées de départ et de fin pour les mouvements, ou l'orientation de départ et de fin pour les rotations. Lorsque l'on a toutes les informations nécessaires, on éxecute la fonction .anima() sur l'animation en question faisant avancer l'animation d'une frame (avec le nombre de frame d'une animation défini dans les valeurs de la classe).

La classe possède aussi 4 constructeurs différents pour éviter de demander toutes les informations à chaque fois, par exemple on n'a pas besoin d'avoir d'informations sur un changement de direction si l'on veut juste faire avancer une cellule.

L'instantiation de la liste d'animations à faire à chaque étape du jeu se fait dans CellType.java et Grid.java tout en mélangeant les animations s'exécutant sur une même cellule par le biais d'une méthode dans AnimaCell à celle déjà présente dans la liste si possible. Ensuite on vas prendre la liste et la parcourir dans GridView avec drawAnimaList, ce qui a pour effet de traverser la liste en utilisant la fonction anima() cité plutôt pour faire progresser toutes les animations jusqu'à leur fin

Contrôleur

GridController, LevelEditorController et SandboxController

La classe GridController permet donc de faire le lien entre le modèle et la vue, ils donnent les évènements que les boutons en jeu doivent faire, elle permet d'actualiser la vue quand le modèle change, de décider de l'arrêt du jeu et d'ajouter le lien avec la classe d'Interaction. Le contrôleur de l'éditeur de niveau est donc héritière de GridController pour changer l'affichage par rapport au mode de jeu de base, le contrôleur du mode sandbox est elle une classe héritière de

LevelEditorController et gère le rajout du joystick et de la limite d'affichage de la grille.

Interaction et InteractionSandbox

Cette classe est donc la classe utilisée pour traduire les interactions que le joueur a avec le jeu, elle gère donc les cliques que le joueur fait sur les cellules du jeu. Elle permet aussi de traiter l'animation de *Drag and drop* car elle est étroitement liée avec les différents évènements que le joueur peut avoir via les cliques.

La classe InteractionSandbox hérite de la classe Interaction mais permet de changer légèrement son comportement pour s'adapter aux grilles potentiellement grandes du mode sandbox.

Gestion des niveaux

LevelLoader

La fonction *levelExist* permet de vérifier si un fichier JSON correspond à un niveau qu'on lui demande. On l'utilise principalement pour l'affichage de l'écran de fin de niveau pour pouvoir afficher ou non le bouton du niveau suivant.

L'autre fonction principale *loadLevel* vérifie si le niveau qu'on veut charger existe bien par sécurité, ensuite on récupère les données stockées dans le fichier JSON correspond pour construire un objet de type Grid.

LoaderJson

Cette classe permet l'écriture et la lecture des fichiers Json des niveaux. Les méthodes *parseGrid*, *parseInteractGrid* et *gridmaker* permettent la lecture des niveaux, elles traitent des données de type JsonArray afin de les renvoyer sous forme de grille de booléen pour *parseInteractGrid* et pour *parseGrid* le fonctionnement diffère, elle renvoie une grille de String qui possède les noms des cellules pour ensuite recréer la grille de type Cell correspondante au niveau.

Les fonctions *writeInFile, interactGridToJSON* et *gridToJSON* permettent d'écrire le contenu d'un niveau en fichier Json pour le fonctionnement de l'éditeur de niveau ou du sandbox. La fonction *gridToJSON* permet de mettre sous la forme d'une JSONArray les informations des cellules du niveau, *interactGridToJSON* permet de faire la même chose mais en stockant les informations de la grille d'interaction. Les informations récupérées par ces méthodes sont ensuite utilisées par *writeInFile* pour les écrire dans un fichier Json.

Gestion de l'audio

Audio et playlist

Audio est une classe dont le rôle est de charger, jouer, stopper, boucler et modifier le volume pour un son par le biais de la classe Clip de javax.sound.sampled. Pour ce faire, on doit d'abord créer une instance d'Audio, puis utiliser la méthode load avec le nom du fichier audio que l'on veut jouer en respectant les spécificités de Clip, puis on peut utiliser les fonctions disponibles (play, loop, stop, setVolume) pour interagir avec le son et close pour décharger l'audio et pouvoir en charger un autre à la place.

Il y a aussi une fonction play statique et prenant en paramètre un chemin d'accès en string en paramètre comme load. Son but est de jouer un son une fois sans sauvegarder d'informations, ce qui est utile pour les SFX sachant qu'un clip ne peux pas être joué plusieurs fois en même temps

La classe Playlist prend une liste d'Audio et un volume en float. Avec la fonction play, on peut jouer les sons de la playlist et de façon aléatoire si on lui passe un booléen en True en paramètre et dans l'ordre si False. De plus avec l'aide de setVolume, on peut modifier le volume des Audios stockées dans l'instance de Playlist. De plus la classe possèdent une fonction statique loadPlayList, prenant en paramètre une liste de String contenant le nom des musiques et renvoyant la liste d'Audio obtenu

Les modes de jeux

Mode spécial

C’est un mode de jeu où avant chaque niveau le joueur peut modifier l’ordre de priorité d’activation des cellules lors des cycles du jeu. Ce qui permet de créer des niveaux encore plus complexes et innovants.

Quand on clique sur un niveau du mode spécial, une fenêtre apparaît. Cette fenêtre contient des boutons à appuyer selon l’ordre de priorité que l'on souhaite. On ne peut revenir sur la fenêtre du jeu tant que l’on a pas fermée celle des priorités.

Son implémentation est très simple. Quand on clique sur le bouton on envoie comme information à LevelLoader qu’on veut charger les niveaux du mode spécial. Quand on clique sur un de ces niveaux, tout se passe comme pour les autres niveaux mais un JDialog apparaît pour nous permettre de changer l'ordre de priorité des cellules. Il suffit de cliquer sur des JButton portant le nom des cellules dans l’ordre souhaité. Chaque clique de bouton ajoute le CellType de la cellule dans un tableau représentant l'ordre des cellules et la validation permet de remplacer le tableau cellOrder par le tableau du JDialog.

Editeur de niveau

L’éditeur de niveau permet au joueur de créer son propre niveau sur une grille de la taille souhaitée. Il peut choisir toutes les cellules implémentées dans le jeu, les faire tourner, les supprimer et choisir si les cases ont une possibilité d’interaction ou non. On peut ensuite sauvegarder le niveau créé et le charger pour ajouter des modifications ou le simuler et donc y jouer.

Sandbox

C’est un mode de jeu qui reprend le mode éditeur. Le principe est d'avoir une très grande grille où l’on peut placer des cellules et les

regarder se comporter. Un joystick permet de se déplacer sur cette grande grille.

Pour cela l'affichage de la grille a été coupé : on définit des bornes de la grille qui seront affichées qui doivent faire maximum une taille de 15 cellules. Le joystick permet donc de bouger les limites de ces bornes pour donner l'illusion que la grille bouge alors que c'est juste la portion de la grille qui est affichée qui change.

Hitman

Le mode de jeu Hitman introduit deux nouveaux types de Cellules: le HITMAN et le BOSS. La particularité de ce Gamemode est que le Boss ne peut être éliminé que par la cellule Hitman. Il faut alors réussir à trouver un moyen d'amener cette cellule jusqu'à celle du Boss.
Le hitman, contrairement aux cellules classiques, ne meurt pas au contact avec une cellule ENEMY, mais seulement lorsqu'il percute la Cellule BOSS.
Les fichiers de ce type de niveau sont stockés dans le dossier hitmanLevels au lieu du dossier regularLevels.

Classe principale

Game

C'est la fonction qui permet le lancement du jeu et qui contient la fonction main, c'est elle qui possède la fenêtre du jeu. Elle permet de stocker les éléments principaux pour permettre le fonctionnement et la navigation entre les différents états du jeu. Elle initialise le CardLayout et certaines fonctions de navigation.

Screen

Cette classe implémente *Runnable* et permet de représenter l'écran du jeu ou on va afficher la grille des cellules, elle est utilisée pour les moments ou on va afficher et jouer au jeu. Elle contient différentes informations importantes pour son fonctionnement comme la grille qu'on veut afficher. Par cette classe on peut changer la taille d'affichage et gérer la fonction principale la fonction *run* qui va permettre le lancement de la boucle du jeu, dedans on effectue les différents changement d'état des cellules.

Conclusion

Répartition du groupe

Hugo :

- Éléments clés du modèle
- Intéractions mères entre cellules (checks de présences, collisions)
- Création des niveaux de tests de cellules
- Affichage de la grille dans un Level
- Lancement d'un Level par lecture Json
- Les Menus
- Effets visuels
- Scaling de plusieurs menus
- Options
- Mode Hitman
- Contrôles dans le mode Sandbox: Joystick V2

Classes éditées:
Game, LevelLoader, Grid, MainMenu, LevelSelectionMenu, RetroGlassPane, LoaderJson, StyleButton, Screen, OptionsMenu, OptionsMenuGridSize, OptionsMenuScreenSize, SandboxController, Interaction, CellType, Direction

Daniel :

- Animations des cellules
- Musique du jeu et SFX avec option de controle de volume
- Environ la moitié des niveaux du jeu
- Fond d'écran des niveaux
- Tiles de fond derrière les cellules (dessiné par lui)
- Texture et affichage ascii
- 1er version de l'affichage final
- Transfert de la liste des textures des cellules vers un json avec lecteur

Classes éditées:
AnimaCell, Audio, Playlist, LevelLoader, CellType, Grid, Direction, GridView, ImageLoader, Render, Texture, OptionMenu, CellTextureLoader, Game, Screen

Elias :

- Comportement de toutes les cellules sauf les spinner, générateurs, et celles du gamemode Hitman
- Mode sandbox
- Mode spécial
- Sauvegarde et chargement de niveau dans les modes Sandbox et éditeur
- Création des niveaux de tests de cellules
- Script pour générer une javadoc
- Pouvoir choisir la taille de la grille dans le mode Sandbox et editeur
- Écran de fin de niveau permettant de passer au niveau suivant si ce niveau existe
- Ajout de nouvelles cellules : nudge, wanderer, repulse, switcher
- Fonctionnement des boutons quand on joue un niveau (play state, restart, play, pause/resume)
- Créations des niveaux du mode spécial.

Classes éditées :
Grid, GridScalePanel, LevelFinish, Screen, ImageLoader, RuleSelectDialog, SandboxMod, SaveDialog, GridScalePanel, LoadFileDialog, InteractionSandbox, SandboxController

Ethan :

- Classes directions et position
- Mode éditeur de niveau
- Comportement des cellules spinner et generator
- Affichage par transformation affine et scaling de la fenêtre dynamique.
- Factorisation et amélioration des classes menus en template
- Fonctions d'écritures en Json
- Création du contrôleur principal (GridController)
- Création de la classe Interaction
- Animation de *Drag and drop*
- Rajouts de niveau
- Classe Cell
- Script de lancement

Classes éditées :

Cell, Direction, Position, CellType, GridController, LevelEditorController, Interaction, LoaderJson, GridView, LevelEditor et tous les fichiers du répertoire view/menu

Baptiste

Problème et solution

Pour résoudre les différents conflits possibles il a été décidé que la cellule se trouvant le plus proche de la coordonnée (0,0) de la grille de cellule agira en première. Et que à chaque appel de la méthode update d'une cellule la grille soit mise à jour pour éviter tout problème possible.

La représentation des différentes cellules du jeu et de leur comportement était un gros défi. Il fallait trouver une représentation appropriée et efficace permettant de facilement donner des comportements aux cellules.

Le fonctionnement de la cellule mover était très complexe à faire et demandait beaucoup de réflexion.

De même pour le fonctionnement de la cellule Wanderer qui était complexe à trouver. Il fallait lui donner un comportement intéressant tout en restant sur cet idée d'une cellule ennemi mouvante. Il fallait que ses déplacements soient semi-déterministes, c'est-à-dire que le joueur pouvait connaître ce qu'elle allait faire même si l'on dirait que la cellule bouge un peu de manière imprévisible. Cette cellule devait aussi paraître intelligente mais pas trop. Elle devait rester tuable.

L'utilisation de la classe Screen permet l'utilisation simplifiée d'une grille pour afficher et de faire tourner la boucle du jeu.

Le choix de factorisation des menus par un fonctionnement via template a été décidé suite à l'apparition de nombreux menus avec une écriture redondante et très similaire entre elles. L'intérêt a été de réduire la quantité de code "copier-coller" en redéfinissant uniquement les différences et en utilisant des énumération pour l'affichage de grande quantité de boutons.

Le choix de stocker les niveaux en format Json permet le stockage par des fichiers stockés dans le jeu. Cela simplifie la lecture et l'écriture des fichiers pour la sauvegarde ou le lancement de partie. Le format Json est un format simple et facile à écrire à la main pour la création de niveau.

La mise en place du mode sandbox posait un défi : la possibilité de créer une grille très grande qui ne pouvait pas tenir dans l'écran. Ce qui a demandé bien sûr la création d'un joystick pour se déplacer et de couper l'affichage de la grille. Ce découpage n'était pas évident. Il fallait rajouter le moins de ligne de code possible.

Conclusion

Ce projet a permis de mettre en pratique les fondamentaux de la programmation orienté objet en java et de l'affichage graphique avec Swing et awt. Nous avons aussi travaillé l'architecture logicielle et en particulier l'architecture MVC, nous avons pu pratiquer chaque étape d'un projet, de la planification à la finalisation du projet via l'outil Git pour le travail d'équipe.

La réflexion apporté pour le développement de ce projet à permis d'apprendre la mise en place concrète de la modularité d'un code avec l'utilisation de polymorphisme comme avec les cellules ou les énumérations de boutons, on peut aussi mentionner l'utilisation de paterne comme celui avec les "template" qui montre une utilisation claire et très pratique de la redéfinition de fonction et d'héritage en Java.

Nous avons aussi pu voir la manipulation d'objet *Runnable* avec l'implémentation pour la class Screen mais aussi pour les expressions lambda utilisé encore une fois dans les énumérations des menus.

L'utilisation plus importante d'interface graphique via Swing et awt nous a appris en plus de l'architecture MVC une utilisation plus complexe de cette librairie avec les transformations affine, la gestion des panel et des *layout*.

La gestion de sauvegarde des niveaux dans les fichiers Json a aussi fait de la pratique de librairie au-delà de celle apprise en cours ce qui a développé des compétences plus concrètes.